Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas

Proyecto fase 3:

Sistema de Gestión de Órdenes y Logística Omnicanal (SGO-Logic)

PONDERACIÓN: 30

Horas Aproximadas: 75

Índice

1. Resumen Ejecutivo **3**
2. Competencias que desarrollaremos **3**
3. Objetivos del Aprendizaje **4**
3.1 Objetivo General 4
3.2 Objetivos Específicos 4
4. Enunciado del Proyecto **5**
4.1 Descripción del problema a resolver 5
4.2 Información del proyecto 6
4.4 Entregables 11
5. Metodología **12**
6. Desarrollo de Habilidades Blandas **13**
6.1 Proyectos en Grupo **13**
7. Cronograma **13**

1. Resumen Ejecutivo

Este proyecto tiene como fin enseñar a los estudiantes a desplegar arquitecturas de software en la nube, usando herramientas y servicios que aseguren escalabilidad, disponibilidad y eficiencia. Aborda la necesidad de desarrollar infraestructuras flexibles y sostenibles en crecimiento. La solución incluye la integración de CI/CD y pruebas de software para garantizar una entrega continua y segura, optimizando el ciclo de vida del software y reforzando la adaptabilidad y estabilidad del sistema y además implementar un nuevo estilo arquitectónico capaz de gestionar las notificaciones y que conviva con la arquitectura ya realizada en las anteriores fases.

2. Competencias que desarrollaremos

- Aplica y adapta modelos de despliegue en la nube mediante estilos arquitectónicos y plataformas como Docker, optimizando la disponibilidad y escalabilidad de los sistemas.
- Implementa y evalúa pruebas funcionales y no funcionales empleando técnicas de pruebas unitarias, de carga y de seguridad (entre otras), asegurando el correcto funcionamiento y desempeño del software.

- Integra nuevos estilos arquitectónicos a la solución diseñada en las fases anteriores.

3. Objetivos del Aprendizaje

3.1 Objetivo General

Que el estudiante aprenda a realizar la integración y despliegue continuo de una arquitectura de software en la nube, utilizando servicios y herramientas de proveedores, con el fin de asegurar escalabilidad, disponibilidad y eficiencia en el uso de recursos, flexibilizando la arquitectura mediante la incorporación de un nuevo estilo arquitectónico que le permita evolucionar sin comprometer lo ya construido en las fases anteriores

3.2 Objetivos Específicos

Al finalizar el proyecto, los estudiantes deberán ser capaces de:

- Explicar los principios de control de versiones y su importancia en la gestión de un proyecto; utilizar herramientas como Git para administrar cambios; diferenciar estrategias de ramificación y flujos de trabajo; valorar los riesgos y beneficios de distintas estrategias colaborativas y diseñar una integración de control de versiones de manera segura, coordinada y eficiente.
- Describir los distintos tipos de pruebas y sus objetivos; implementar pruebas unitarias, de integración y de aceptación en módulos específicos; comparar resultados para identificar errores críticos; determinar la efectividad de una estrategia de pruebas en la calidad del producto entregado; y crear un plan integral de pruebas que garantice la calidad general de la arquitectura.
- Explicar los conceptos de CI/CD y su relevancia en la entrega continua; configurar pipelines automatizados de integración y entrega de código; examinar su impacto en escalabilidad, resiliencia y reducción de riesgos; justificar el uso de CI/CD frente a metodologías tradicionales; y diseñar e implementar una estrategia de CI/CD que estandarice y automatice las etapas de desarrollo, pruebas y despliegue mediante el uso de las herramientas propuestas.

- Recordar y comprender los principios de una arquitectura de software y su aplicación en la evolución de un sistema ya construido, analizar cómo una buena definición arquitectónica permite incorporar una nueva regla de negocio, la evaluación de crédito, sin comprometer la escalabilidad ni la resiliencia del sistema existente, y finalmente crear una arquitectura que optimice la evaluación de elegibilidad y la coordinación entre las rutas de

pago tradicional y crédito propio, garantizando rapidez, confiabilidad y eficiencia en la decisión y en la notificación al cliente.

4. Enunciado del Proyecto

4.1 Descripción del problema a resolver

Con el MVP de SGO-Logic ya operando y validado, la Gerencia General convoca al equipo de arquitectos a una reunión extraordinaria. El negocio quiere abrir una nueva línea de ingresos antes de que termine el trimestre, y quiere hacerlo sin detener lo que ya está funcionando.

La gerente de marketing María José Rivas estableció que: "Vamos a lanzar una campaña de crédito propio: 'Compra ahora, paga en 30 días'. Esto nos diferenciará de la competencia y aumentará el ticket promedio en un 40%. Pero necesitamos que el sistema se adapte rápido."

La propuesta comercial implica que cuando un cliente llegue al checkout, ya no bastará con decidir *si puede pagar*; el sistema deberá decidir primero *cómo* va a pagar. Para algunos clientes, la opción de crédito propio aparecerá como una alternativa más junto a PayKlik; para otros, simplemente no existirá, porque el negocio no está dispuesto a asumir el riesgo de un cliente con mal comportamiento de pago.

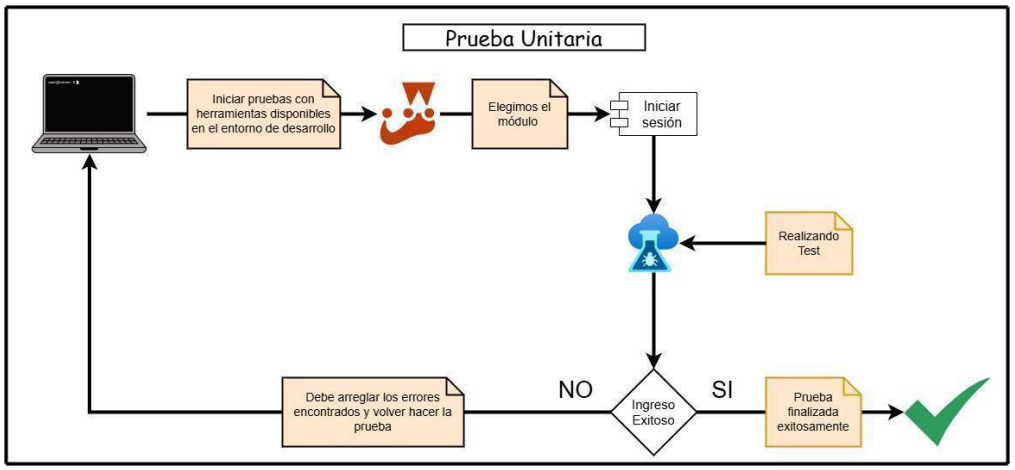
Esa distinción no puede quedar en manos de un agente de atención ni resolverse revisando historiales a mano como ocurría antes. Finanzas establece que no están dispuestos a otorgar crédito sin criterio: cada solicitud tendría que sustentarse en el comportamiento real de compra del cliente, cuánto ha comprado, qué tan puntual ha sido, si acumula devoluciones o incidentes. Esta información se encuentra dispersa en el ERP heredado y nunca se ha usado para decidir nada en el instante mismo de la venta.

El análisis de viabilidad del crédito debe ser certero y de forma rápida. La orden no puede quedar congelada en el checkout esperando una respuesta. Sea cual sea la ruta elegida ya sea crédito propio o el pago tradicional ya conocido, la orden debe terminar en el mismo lugar: confirmada o rechazada, con el cliente enterado de qué pasó con su solicitud, sin que ninguna de las dos rutas se convierta en un obstáculo para la otra.

Esta ampliación de alcance debe tratarse como una extensión disciplinada de la arquitectura ya definida en las Fases 1 y 2, no como un módulo aislado. Se espera que el equipo identifique qué nuevos actores, drivers, escenarios de calidad y decisiones arquitectónicas surgen de esta declaración de negocio, y que actualice el DDA y los diagramas correspondientes.

Pruebas

1. **Pruebas unitarias**
   Las pruebas unitarias se pueden definir simplemente como un procedimiento que se ejecuta para validar que cada unidad o componente individual de una aplicación funcione correctamente. Este tipo de pruebas confirman que cada módulo en aislamiento produce el resultado esperado dado un conjunto de entradas. Por ende, se verifican partes específicas del código sin interacción con dependencias externas. Tiene opciones a utilizar como Jest, Mocha o cualquier Framework que se adapte a su proyecto, para realizar pruebas unitarias y asegurar que cada función individual trabaje según lo esperado. Deberá implementar un mínimo de 5 pruebas para cubrir todos los escenarios posibles.



2. **Pruebas de integración**
   Las pruebas de integración se pueden definir simplemente como un procedimiento que se ejecuta para verificar la interacción entre diferentes módulos o componentes de una aplicación. Este tipo de pruebas confirman que los módulos integrados funcionan juntos como se espera, detectando posibles errores en las interfaces y flujos de datos entre ellos. Por ende, se valida la correcta comunicación y coordinación entre los componentes que se combinan para realizar funciones más complejas. Tiene opciones de herramientas como Jest, Mocha, pytest o cualquier otra herramienta para realizar pruebas de integración, asegurando que las distintas partes del sistema se integren de manera fluida y correcta. Deberá implementar un mínimo de 5 pruebas.

3. **Pruebas de carga**
   Las pruebas de carga se pueden definir como un procedimiento que se ejecuta para evaluar el comportamiento de una aplicación cuando se somete a una carga específica de usuarios, transacciones o peticiones simultáneas.

Este tipo de pruebas confirman que el sistema mantiene un desempeño aceptable bajo condiciones esperadas de uso, verificando tiempos de respuesta, consumo de recursos y estabilidad general. Por ende, se evalúa la capacidad del software para operar de manera eficiente en escenarios que reflejan la demanda real de producción. Deberá implementar un mínimo de 5 pruebas de carga.

4. **Pruebas de estrés**
   Las pruebas de estrés se pueden definir como un procedimiento que se ejecuta para evaluar el comportamiento de una aplicación cuando se somete a condiciones extremas de carga, superiores a las esperadas en su operación normal. Este tipo de pruebas confirman cómo responde el sistema ante picos inesperados de demanda, identificando puntos de falla, cuellos de botella y límites de capacidad. Por ende, se evalúa la resiliencia del software, su capacidad de recuperación y la estabilidad posterior a situaciones de sobrecarga. Deberá implementar un mínimo de 5 pruebas de estrés.

5. **Pruebas E2E**
   Las pruebas end-to-end (E2E) son un tipo de prueba que valida todo el flujo de un sistema de principio a fin, asegurando que todos los componentes, módulos y dependencias externas funcionan juntos como se espera en un entorno lo más cercano posible a producción.

Despliegue

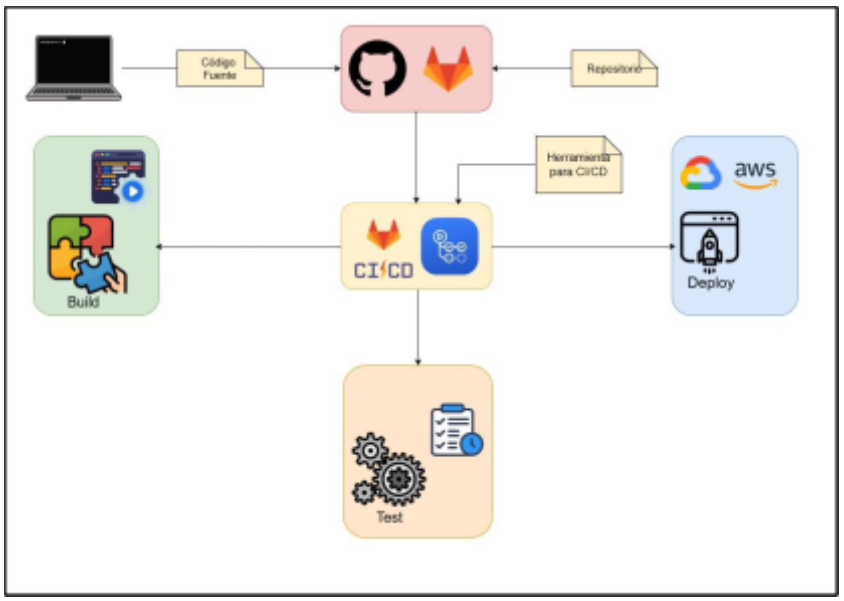
Uso de CI/CD

Para lograr un despliegue eficiente y escalable del proyecto, se implementará un proceso de Integración Continua/Despliegue Continuo (CI/CD). Este proceso asegura que cada cambio en el código sea integrado, probado, y desplegado de manera automatizada, reduciendo errores humanos y acelerando la entrega de nuevas versiones del software. A continuación, se describen los tres pipelines clave que se utilizarán en este proceso:

- **Build**
  En esta etapa, el código fuente se compila y se generan los artefactos necesarios para el despliegue, asegurando que el código sea válido y esté listo para ser probado.
- **Test**
  Durante esta fase, se ejecutan pruebas automatizadas para validar la funcionalidad del código. Esto incluye **pruebas unitarias y de integración**, para garantizar la calidad del software.

  No se debe hacer deploy si las pruebas no pasan
- **Deploy**
  En esta etapa final, el código que ha pasado las pruebas es desplegado en los entornos de producción o de pruebas. Este paso puede incluir el despliegue a servidores, contenedores, o servicios en la nube, garantizando que la aplicación esté disponible para los usuarios finales.

A continuación, se presenta un ejemplo de cómo sería el uso de CI/CD.
IMPORTANTE: ESTA IMAGEN ES DE REFERENCIA Y NO REPRESENTA LA MANERA EN QUE SE DEBE IMPLEMENTAR EL CI/CD EN LA FASE 3.



La herramienta para el uso de CI/CD será GitLab o Github Actions

Docker

Permite crear, desplegar y gestionar aplicaciones en contenedores, los cuales son entornos de ejecución aislados que incluyen todo lo necesario para que una aplicación funcione: el código, las bibliotecas, las dependencias, y el sistema operativo base. A diferencia de las máquinas virtuales, los contenedores son más livianos y se inician mucho más rápido, ya que comparten el núcleo del sistema operativo subyacente.

Docker es fundamental en una arquitectura de software, ya que permite crear contenedores que garantizan la consistencia de las aplicaciones en diferentes entornos, eliminando problemas de compatibilidad y simplificando tanto el desarrollo como el despliegue. Al encapsular todas las dependencias necesarias, Docker asegura que las aplicaciones funcionen de manera idéntica en desarrollo, pruebas y producción, y proporciona portabilidad al permitir que los contenedores se ejecuten en cualquier sistema con Docker. Además, facilita la escalabilidad y la gestión eficiente de recursos al permitir la ejecución de múltiples contenedores en un solo servidor, optimizando así el uso de recursos y mejorando la capacidad de respuesta ante la demanda.

Para esta fase se le solicita al estudiante que utilice Docker tanto en el Backend como en el Frontend. Debe incluirse en el proceso de CI/CD.

4.4 Entregables

- **Modificación de la documentación de la fase 1 y 2 de acuerdo al feedback dado en las revisiones.**
- **Diseño e implementación de la nueva arquitectura.**
  - Mínimo 4 releases
- **CI/CD**
  - Utilizar GitLab o Github para hacer las fases de DevOps (build, test, deploy), desplegando a la nube su arquitectura documentada en las fases anteriores.
- **Pruebas (5 pruebas mínimo por cada tipo, para un total de 25 pruebas mínimo)**
  - Unitarias
  - Integración
  - End to End
  - Estrés
  - Cargas

Documentación

  - Manual Técnico
  - Manual de usuario

CONSIDERACIONES:

- Es obligatorio el uso de una estrategia de **git-flow** como estrategia de branching
  - Durante el desarrollo de la fase debe hacer un mínimo de 4 releases
- Todos los integrantes deben estar presentes el día de la calificación, ya que se validará la participación de los integrantes.
- Agregar al aux de su sección correspondiente desde el primer día después de la publicación del enunciado:

  - Kevin Hernández Sección A (Kjhg4589)
  - Juan Carlos Aragón Sección B (aragonjc)
  - Abdiel Fernando Otzoy Otzín Sección B (AbdielOtzoy)

- La documentación debe ser almacenada en el repositorio utilizando el formato MarkDown
- **Las copias parciales o totales tendrán una nota de 0 y será reportada a la escuela de sistemas.**
- La calificación obtenida puede cambiar si la ingeniera solicita una revisión.

- Todos los integrantes del grupo deberán tener commits sustanciales a lo largo del desarrollo. Utilizar el formato de commits para dejar prueba de su trabajo. Hacer merge de pull requests no cuenta como commit válido.
- Documentación formato markdown en un repositorio de github y código de la solución
- Link al mismo repositorio de las fases anteriores a través de UEDI

5. Metodología

Se debe realizar la estrategia de branching git-flow, en donde se encuentran las ramas principales llamadas main y develop. Por otra parte, están las ramas de soporte que son feature, hotfix, etc. Cada commit debe llevar el formato #carnet: mensaje (eg., 12345678: agregar botón). Para cada feature se hace una rama a partir de develop, y al terminar la feature, se hace merge a develop y se elimina la rama feature creada. Al finalizar el sprint, todas las features implementadas se encuentran en develop, y se procede a hacer merge a main y se hace un nuevo release a través de un tag. El formato del tag deberá ser: v1, v2, v3, etc.

6. Desarrollo de Habilidades Blandas

Para complementar el desarrollo técnico, esta sección se centra en las habilidades blandas que los estudiantes deberán mejorar a lo largo del proyecto, como la comunicación, el liderazgo, y la colaboración en equipo.

6.1 Proyectos en Grupo

El trabajo en equipo es clave para el éxito de cualquier proyecto. Los estudiantes deben colaborar, asignando roles específicos (líder de proyecto, desarrollador, diseñador, etc.) para organizar y distribuir tareas de manera eficiente. A través de herramientas como Trello, Google Docs o GitHub, los equipos gestionarán sus avances y compartirán información de manera eficaz.

Además, se fomenta la toma de decisiones conjunta y la resolución de problemas en grupo, desarrollando habilidades como la comunicación, el liderazgo y la negociación. Los grupos recibirán retroalimentación constante para mejorar sus soluciones y adaptarse a los desafíos que surjan.

6.1.1 Trabajo en Equipo

Cada grupo debe trabajar en colaboración, asignando roles específicos a cada miembro (líder de proyecto, desarrollador, diseñador, etc.). Los estudiantes deben coordinarse mediante herramientas de gestión como Trello, Google Docs o GitHub.

6.1.2 Comunicación Efectiva

Los grupos deben realizar presentaciones periódicas ante sus compañeros y el

profesor, compartiendo los avances y problemas encontrados, y recibiendo retroalimentación para mejorar sus soluciones.

6.1.3 Resolución de Conflictos

A lo largo del proyecto, los estudiantes pueden enfrentar desafíos o desacuerdos. Deberán aplicar técnicas de resolución de conflictos para mantener la armonía y la productividad en el equipo.

7. Cronograma

Tipo	Fecha
Asignación de Proyecto	24 de Septiembre 2026
Entrega Final	14 de Octubre 2026
Calificación	17 de Octubre 2026

En el desarrollo del proyecto, se espera que cada estudiante demuestre honestidad académica y profesionalismo. Por lo tanto, se establecen los siguientes principios:

1. **Originalidad del Trabajo**
   - Cada estudiante o equipo debe desarrollar su propio código y/o documentación, aplicando los conocimientos adquiridos en el curso.
2. **Prohibición de Copias y Plagio**
   - Si se detecta la copia total o parcial del código, documentación o cualquier otro entregable, la calificación será de **0 puntos**.
   - Esto incluye la reproducción de código entre compañeros, la reutilización de proyectos de semestres anteriores o el uso de código externo sin la debida referencia.
3. **Uso Responsable de Recursos Externos**
   - El uso de bibliotecas, frameworks y ejemplos de código externos está permitido, siempre y cuando se referencian correctamente y se comprendan plenamente. (Consultar con el catedrático su política)
4. **Revisión y Detección de Plagio**
   - Se podrán utilizar herramientas automatizadas y revisiones manuales para identificar similitudes en los proyectos.
   - En caso de sospecha, el estudiante deberá justificar su código y demostrar su desarrollo individual o en equipo. Si este extremo no es comprobable la calificación será de **0 puntos**.

Al detectarse estos aspectos se informará al catedrático del curso quien realizará las acciones que considere oportunas.